

误差分析

本节比较 Black-Scholes-Merton 模型 (BSM, 常数波动率) 与 Heston 随机波动率模型对同一份 chooser 期权的定价。chooser 为场外产品, 基准价格由与普通期权市场价格相一致的 Heston 模型给出: 以 2018–2024 年 84 个月度期权链快照 (每月首个交易日) 逐一校准 Heston 参数, 拟合的隐含波动率 RMSE 均值为 1.10 个波动率点; 快照之间的交易日沿用最近快照的 $\kappa, \theta, \xi, \rho, v_0$ 按 VIX 的变化缩放。样本为 2018-01-02 至 2024-12-31 共 1761 个交易日, 每日对两份合约定价: $K = S_t$ (固定 moneyness) 与 $K = 150$, $T_1 = 0.5$, $T_2 = 1$ 。Heston 价格采用 QE 格式的蒙特卡罗 (每日 10 万条路径, $[0, T_1]$ 与 $[T_1, T_2]$ 各 32 步), 并与特征函数积分得到的半解析解逐日对照。BSM 的波动率取两种设定: σ_{hist} 为 20 日历史波动率, σ_{impl} 为 Heston 在期限 T_2 、行权价等于远期价格处的隐含波动率。

记 $\tau = T_2 - T_1$, $K' = Ke^{-(r-q)\tau}$ 。在 r, q 为确定性常数的条件下, chooser 的分解公式与模型无关, 对任一模型 M 有

$$V^M = C^M(K, T_2) + e^{-q\tau} P^M(K', T_1).$$

因此 BSM 与 Heston 的价差等于两条腿价差之和:

$$\Delta V = V^{\text{BSM}} - V^{\text{H}} = [C^{\text{BSM}} - C^{\text{H}}](K, T_2) + e^{-q\tau} [P^{\text{BSM}} - P^{\text{H}}](K', T_1).$$

记 σ_C, σ_P 为 Heston 两条腿价格对应的 BSM 隐含波动率, 即 $C^{\text{BSM}}(K, T_2; \sigma_C) = C^{\text{H}}(K, T_2)$, $P^{\text{BSM}}(K', T_1; \sigma_P) = P^{\text{H}}(K', T_1)$ 。由中值定理,

$$\Delta V = \nu_C(\sigma - \sigma_C) + e^{-q\tau} \nu_P(\sigma - \sigma_P),$$

其中 ν_C, ν_P 为两条腿的 vega, σ 为 BSM 采用的常数波动率。两条腿位于波动率曲面的不同位置, Call 腿在 (K, T_2) , Put 腿在 (K', T_1) , Heston 在两处给出不同的隐含波动率, BSM 对两处使用同一个 σ , 价差由此产生。

对 σ_{hist} 版本, 总误差进一步分解为波动率估计误差与曲面形状误差:

$$V^{\text{BSM}}(\sigma_{\text{hist}}) - V^{\text{H}} = \underbrace{V^{\text{BSM}}(\sigma_{\text{hist}}) - V^{\text{BSM}}(\sigma_{\text{impl}})}_{\approx (\nu_C + e^{-q\tau} \nu_P)(\sigma_{\text{hist}} - \sigma_{\text{impl}})} + \underbrace{V^{\text{BSM}}(\sigma_{\text{impl}}) - V^{\text{H}}}_{\text{曲面形状误差}}.$$

第一项反映历史波动率与市场隐含波动率的差异, 第二项在两个模型的波动率水平对齐之后保留下来, 反映常数波动率假设对偏斜与期限结构的忽略。

误差指标以 $e_t = V_t^{\text{BSM}} - V_t^{\text{H}}$ 为基础 ($e_t > 0$ 表示 BSM 高估), 包括

$$\text{MAE} = \frac{1}{n} \sum_t |e_t|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_t e_t^2}, \quad \text{Bias} = \frac{1}{n} \sum_t e_t, \quad \text{MAPE} = \frac{10}{n}$$

每项指标分别在全样本与两个子样本上计算。可靠子样本剔除校准参数触及搜索边界或拟合 RMSE 超过 2 个波动率点的快照 (9 个快照, 占交易日 10.7%), 以及 v_0 触及上界的

日期；第二个子样本在此基础上再剔除 120–450 天合约的拟合 RMSE 超过 1.5 个波动率点的快照。

基准价格含三类不确定性：蒙特卡罗的统计误差，由标准误 $se = s/\sqrt{N}$ 度量；时间离散的系统偏差，由蒙特卡罗与半解析解之差的均值度量；Heston 对市场期权的校准误差，折算为价格约为 $(\nu_C + e^{-qT}\nu_P) \times \text{RMSE}_{iv}$ ，其中 RMSE_{iv} 取两条腿所在期限（120–450 天）的合约。BSM 与 Heston 的价差以这三项为尺度进行解读，超出该尺度的部分归因于模型差异。

1 基准的可靠性

```
In [1]: import sys
from pathlib import Path

ROOT = next(p for p in [Path.cwd(), *Path.cwd().parents] if (p / "co_bsm").is_dir)
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import cm
from scipy.stats import norm, rankdata

from co_bsm import calibrate as cal
from co_bsm.black_scholes import BSMPParams, bsm_call, bsm_put
from co_bsm.chooser import ChooserParams, chooser_price
from co_bsm.implied_vol import implied_vol
from co_bsm.pricing import HestonPricer
from co_data_center.data_center import DataCenter
from common.paths import get_outputs_path

plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "PingFang SC",
                                   "Noto Sans CJK SC", "WenQuanYi Zen Hei", "DeJ"]
plt.rcParams["axes.unicode_minus"] = False
pd.set_option("display.float_format", "{:.4f}".format)
pd.set_option("display.width", 200)

out = get_outputs_path()
prices = pd.read_parquet(out / "chooser_daily_prices.parquet")
params = pd.read_parquet(out / "heston_daily_params.parquet")[["date", "snapshot"]]
feat = DataCenter().get("feature_daily").df
feat = feat.assign(date=pd.to_datetime(feat["date"]))[["date", "vix", "sentiment"]]

df = prices.merge(params, on="date", how="left").merge(feat, on="date", how="left")
df["reliable"] = ~df["low_conf"] & (df["v0"] < 0.999)  # v0 被截断在上界的日
BENCH = "heston_mc"

print(f"{df['date'].nunique()} 个交易日，{len(df)} 行；可靠的行占 {df['reliable'].sum()/len(df)*100}%")
print("MC 列是否完整：", df[BENCH].notna().all())
```

2026-10-06 18:31:45 [INFO] co_data_center: feature_daily: 已加载 (DataObject, 6763 行)

1761 个交易日，3522 行；可靠的行占 89.3%

MC 列是否完整： True

1.1 统计误差

```
In [2]: mc = df.dropna(subset=[BENCH, "heston_mc_se"]).copy()
mc["z"] = (mc[BENCH] - mc["heston_exact"]) / mc["heston_mc_se"]
mc["rel_se_pct"] = mc["heston_mc_se"] / mc["heston_exact"] * 100

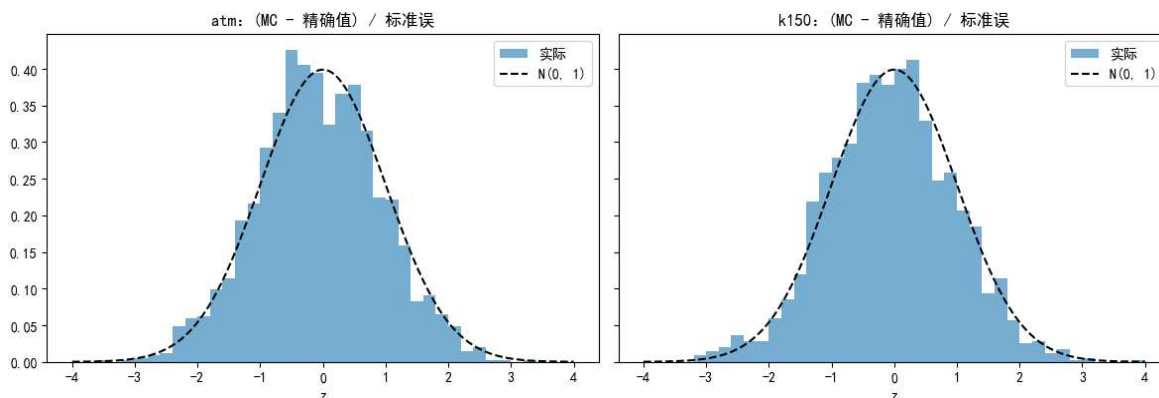
stat = mc.groupby("contract").agg(
    天数=("z", "size"),
    标准误中位数=("heston_mc_se", "median"),
    相对标准误中位数_pct=("rel_se_pct", "median"),
    z均值=("z", "mean"),
    z标准差=("z", "std"),
    超过2的占比=("z", lambda s: (s.abs() > 2).mean()),
    超过3的占比=("z", lambda s: (s.abs() > 3).mean()),
)

print("纯抽样噪声下: z 均值 0, 标准差 1, 超过 2 的占比 4.6%, 超过 3 的占比 0.27%")
display(stat)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.2), sharey=True)
x = np.linspace(-4, 4, 200)
for ax, (name, g) in zip(axes, mc.groupby("contract")):
    ax.hist(g["z"], bins=40, range=(-4, 4), density=True, alpha=0.6, label="实际")
    ax.plot(x, norm.pdf(x), "k--", label="N(0, 1)")
    ax.set(title=f"{name}: (MC - 精确值) / 标准误", xlabel="z")
    ax.legend()
plt.tight_layout()
plt.show()
```

纯抽样噪声下: z 均值 0, 标准差 1, 超过 2 的占比 4.6%, 超过 3 的占比 0.27%

	天数	标准误中位数	相对标准误中位数_pct	z均值	z标准差	超过2的占比	超过3的占比
contract							
atm	1761	0.0685	0.2914	-0.0656	0.9815	0.0460	0.0017
k150	1761	0.0705	0.2101	-0.0541	1.0096	0.0460	0.0034



统计误差. 每日 10 万条路径下, 蒙特卡罗价格的标准误中位数为 0.069 ($K = S_t$) 与 0.071 ($K = 150$), 分别占价格的 0.29% 与 0.21%。 $z_t = (\hat{V}_t - V_t^{\text{exact}})/\text{se}_t$ 的标准差为 0.98 与 1.01, 绝对值超过 2 的占比均为 4.6%, 超过 3 的占比为 0.17% 与 0.34%, 与标准正态分布的 4.6% 和 0.27% 吻合, 所报告的标准误如实反映了抽样误差。 z_t 的均值为 -0.066 与 -0.054 , 对应下一节的平均偏差。

1.2 离散误差

```
In [3]: mc["bias"] = mc[BENCH] - mc["heston_exact"]
mc["regime"] = np.select([mc["xi"] >= 3.9, mc["kappa"] >= 24.9], ["ξ 撞界", "κ 撞界"])

def bias_table(keys):
    rows, index = [], []
    for k, g in mc.groupby(keys):
        n = len(g)
        mean = g["bias"].mean()
        se = np.sqrt((g["heston_mc_se"] ** 2).sum()) / n # 各天的 MC 相互独立
        rows.append({"天数": n, "平均偏差": mean, "均值标准误": se, "偏差/标准误": mean / se,
                     "平均相对偏差_pct": (g["bias"] / g["heston_exact"]).mean()})
        index.append(k)
    if isinstance(keys, list):
        return pd.DataFrame(rows, index=pd.MultiIndex.from_tuples(index, names=keys), name="bias_table")
    return pd.DataFrame(rows, index=pd.Index(index, name=keys), name="bias_table")

print("按合约: ")
display(bias_table("contract"))
print("按参数状态: ")
display(bias_table(["contract", "regime"]))
print("按可靠性: ")
display(bias_table(["contract", "reliable"]))
```

按合约:

	天数	平均偏差	均值标准误	偏差/标准误	平均相对偏差_pct
contract					
atm	1761	-0.0052	0.0017	-3.0597	-0.0181
k150	1761	-0.0030	0.0019	-1.5444	-0.0111

按参数状态:

		天数	平均偏差	均值标准误	偏差/标准误	平均相对偏差_pct
contract	regime					
atm	κ 撞界	19	-0.0424	0.0197	-2.1481	-0.1473
	ξ 撞界	147	-0.0138	0.0066	-2.0982	-0.0482
	其他	1595	-0.0039	0.0017	-2.2456	-0.0138
k150	κ 撞界	19	0.0056	0.0277	0.2033	0.0103
	ξ 撞界	147	-0.0116	0.0071	-1.6435	-0.0319
	其他	1595	-0.0023	0.0020	-1.1450	-0.0094

按可靠性:

		天数	平均偏差	均值标准误	偏差/标准误	平均相对偏差_pct
contract	reliable					
atm	False	188	-0.0119	0.0058	-2.0404	-0.0399
	True	1573	-0.0044	0.0018	-2.4821	-0.0155
k150	False	188	-0.0081	0.0065	-1.2528	-0.0232
	True	1573	-0.0024	0.0020	-1.1752	-0.0097

离散误差. 每日的蒙特卡罗价格与精确值之差取平均后，抽样噪声互相抵消，剩下的是时间离散带来的系统偏差。平均偏差为 -0.005 ($K = S_t$) 与 -0.003 ($K = 150$)，分别是均值标准误的 3.1 倍与 1.5 倍，占价格的 0.018% 与 0.011%。参数触及边界的日期偏差更大： $K = S_t$ 合约在 κ 触界的 19 天为 -0.042 (0.15%)，在 ξ 触界的 147 天为 -0.014 ； $K = 150$ 合约对应为 $+0.006$ (不显著) 与 -0.012 。可靠日期的平均偏差为 -0.004 ，不可靠日期为 -0.012 ($K = S_t$)。步长检验在四组校准得到的真实参数上进行：每日定价采用的每段 32 步下，QE 的偏差在 -0.04 至 $+0.04$ 之间，不超过 1.0 个标准误；Full Truncation 的偏差为 $+0.21$ 至 $+1.06$ ，为 4.9 至 12.8 个标准误，因此基准采用 QE。

1.3 校准误差

```
In [4]: PN = ["v0", "kappa", "theta", "xi", "rho"]
hp = pd.read_parquet(out / "heston_params.parquet")
hp = hp[hp["status"] == "ok"].set_index("date")
chain, featc = cal.load_inputs()

def iv_surface(p, ks, Ts):
    """Heston 隐含波动率，只取决于  $\ln(K/F)$  和期限，与  $S$ 、 $r$ 、 $q$  无关。返回  $(\text{len}(Ts))$ 
     $S = 100.0$ 
     $K = S * \exp(ks)$ 
     $otm = ks >= 0$ 
     $res = \text{np.empty}((\text{len}(Ts), \text{len}(ks)))$ 
    for i, T in enumerate(Ts):
        c = HestonPricer(S, 0.0, 0.0, np.full(len(K), T), K).call_prices(**p)
        res[i] = implied_vol(np.where(otm, c, c - (S - K)), S, K, 0.0, T, 0.0, c)
    return res

def snapshot_options(date):
    S, r, q = cal.market_inputs(featc, date)
    opts, _ = cal.select_options(chain[chain["date"] == date], S, r, q)
    opts["k"] = np.log(opts["K"] / (S * np.exp((r - q) * opts["T"])))
    return opts
```

```
2026-10-06 18:31:45 [INFO] co_data_center: raw_option_chain: 已加载 (OptionChainObject, 91778 行)
2026-10-06 18:31:45 [INFO] co_data_center: feature_daily: 已加载 (DataObject, 6763 行)
```

```
In [5]: def plot_snapshot(date, n_curves=6):
date = pd.Timestamp(date)
row = hp.loc[date]
```

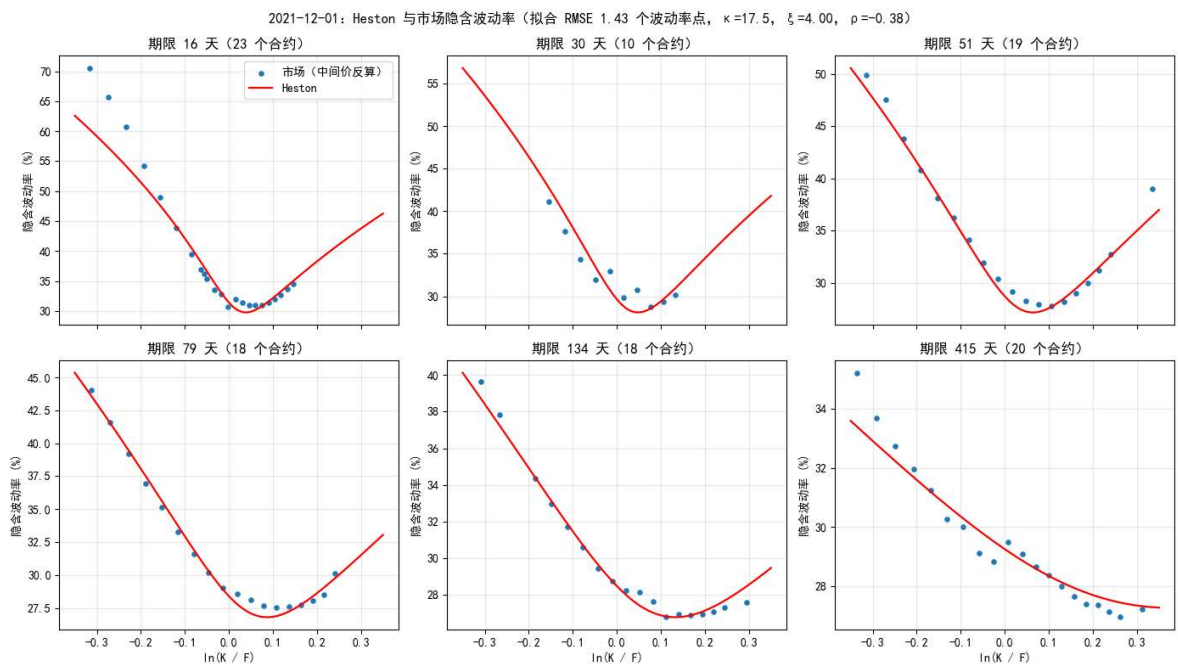
```

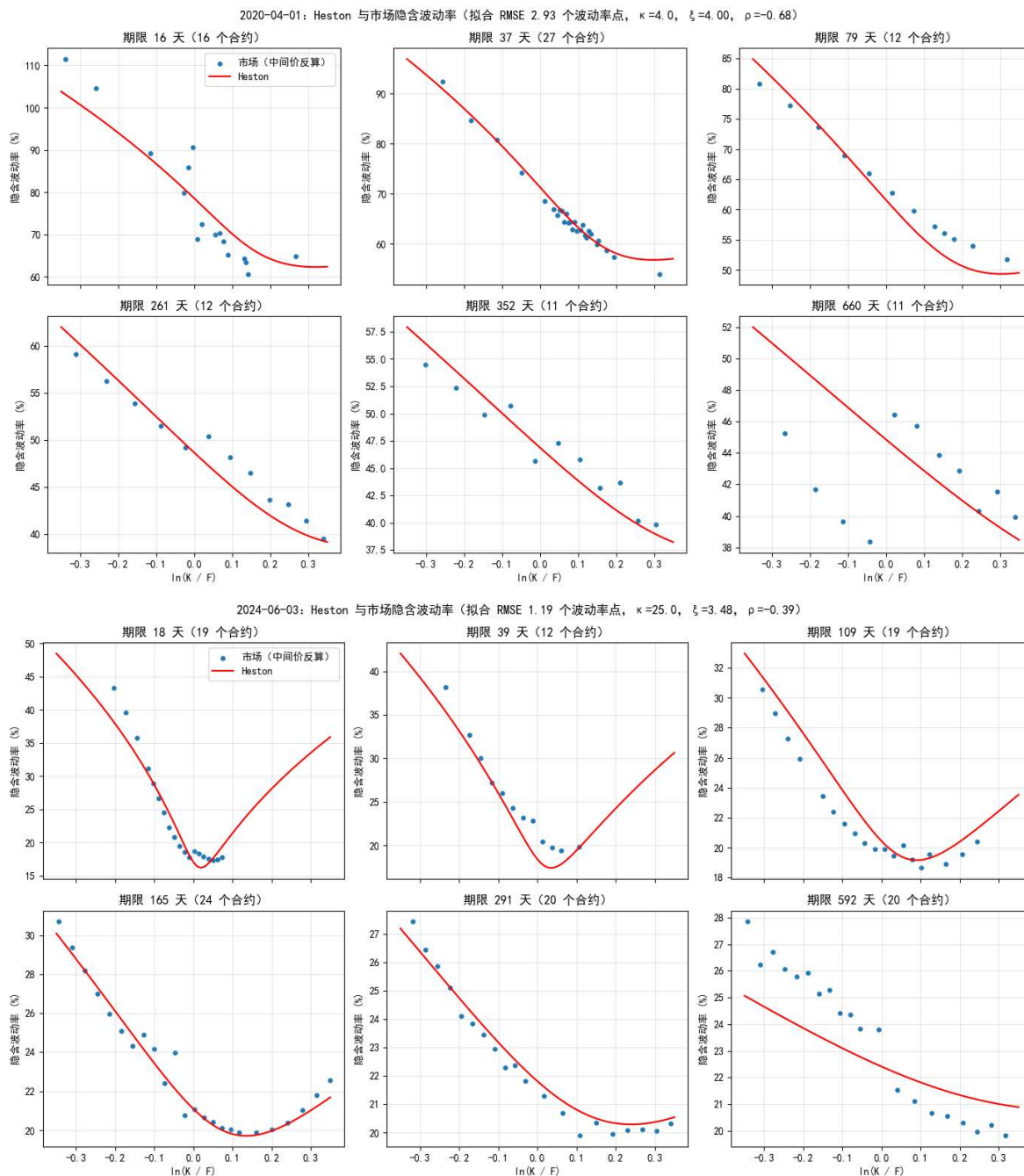
p = {k: float(row[k]) for k in PN}
opts = snapshot_options(date)

Ts = np.sort(opts["T"].unique())
pick = Ts[np.unique(np.linspace(0, len(Ts) - 1, n_curves).round()).astype(int)]
ks = np.linspace(-0.35, 0.35, 71)
fig, axes = plt.subplots(2, 3, figsize=(14, 8), sharex=True)
for ax, T in zip(axes.flat, pick):
    m = opts[np.isclose(opts["T"], T)]
    ax.scatter(m["k"], m["iv"] * 100, s=14, label="市场（中间价反算）")
    ax.plot(ks, iv_surface(p, ks, np.array([T]))[0] * 100, "r-", lw=1.6, label="Heston")
    ax.set(title=f"期限 {T * 365:.0f} 天（{len(m)} 个合约）", ylabel="隐含波动率 (%)")
    ax.grid(alpha=0.3)
axes[0, 0].legend()
for ax in axes[-1]:
    ax.set_xlabel("ln(K / F)")
fig.suptitle(f"{date.date()}: Heston 与市场隐含波动率（拟合 RMSE {row['rmse_']:.2f}, "
             f" $\kappa={p['kappa']:.1f}$ ,  $\xi={p['xi']:.2f}$ ,  $\rho={p['rho']:.2f}$ ）")
plt.tight_layout()
plt.show()

for dt in ["2021-12-01", "2020-04-01", "2024-06-03"]:
    plot_snapshot(dt)

```





```
In [6]: def fit_errors(date):
    """某个快照上各合约的拟合误差，单位为波动率点（模型 - 市场）"""
    p = {k: float(hp.loc[date, k]) for k in PN}
    opts = snapshot_options(date)
    model = np.empty(len(opts))
    for T in opts["T"].unique():
        m = np.asarray(np.isclose(opts["T"], T))
        model[m] = iv_surface(p, opts.loc[m, "k"].to_numpy(), np.array([T]))[0]
    return pd.DataFrame({"date": date, "days": opts["T"].to_numpy() * 365, "k":
        "err": (model - opts["iv"].to_numpy()) * 100})

def rmse(s):
    return np.sqrt((s ** 2).mean())

errs = pd.concat([fit_errors(dt) for dt in hp.index], ignore_index=True)

# 重算的 RMSE 应与校准时记录的一致
```



```
def chooser_at(sig):
    return chooser_price(ChooserParams(S=d["S"].to_numpy(), K=d["K"].to_numpy(),
                                        T1=d["T1"].to_numpy(), T2=d["T2"].to_numpy(),
                                        sigma=sig, q=d["q"].to_numpy()))

s = d["sigma_impl"].to_numpy()
d["vega_per_pt"] = chooser_at(s + 0.005) - chooser_at(s - 0.005) # sigma 变
d["calib_unc"] = d["vega_per_pt"] * d["rmse_rel"]
d["gap_impl"] = (d["bsm_impl"] - d[BENCH]).abs()
d["gap_gt_unc"] = d["gap_impl"] > d["calib_unc"]

display(d.groupby("contract").agg(
    相关期限RMSE_点=("rmse_rel", "median"),
    chooser_vega每点=("vega_per_pt", "median"),
    校准不确定性_中位数=("calib_unc", "median"),
    MC标准误_中位数=("heston_mc_se", "median"),
    BSM与Heston绝对差_中位数=("gap_impl", "median"),
    差距大于校准不确定性的占比=("gap_gt_unc", "mean")))
```

	撞界	未撞界
count	8.0000	76.0000
mean	0.8400	0.7000
std	0.6000	0.3600
min	0.4300	0.2600
25%	0.5500	0.5100
50%	0.6500	0.5900
75%	0.7700	0.7500
max	2.2700	2.0700

120-450 天 RMSE 超过 1.5 的快照: {'2020-04-01': 2.27, '2020-05-01': 1.87, '2020-06-01': 1.98, '2020-07-01': 1.65, '2020-08-03': 2.07, '2020-09-01': 1.67}
其中已标为低可信度: ['2020-04-01']

	相关期限 RMSE_点	chooser_vega 每点	校准不确 定性_中位 数	MC标准 误_中位 数	BSM与 Heston绝对 差_中位数	差距大于校 准不确定性的 占比
contract						
atm	0.6078	0.8613	0.5823	0.0685	0.3581	0.2993
k150	0.6078	0.4878	0.3891	0.0705	0.9509	0.8132

校准误差. 84 个快照共 19940 个参与校准的合约, Heston 拟合隐含波动率的 RMSE 为 1.10 个波动率点。按期限分层, RMSE 呈两头高、中间低: 期限短于 45 天为 1.41 (占合约数的 42%), 45-120 天为 1.16, 120-240 天为 0.80, 240-450 天为 0.75, 长于 450 天为 1.18。chooser 的两条腿分别位于约 182 天 ($T_1 = 0.5$) 与 365 天 ($T_2 = 1$), 落在拟合最好的区间; 各快照在 120-450 天合约上的 RMSE 均值为 0.72、中位数为 0.60、最大值为 2.27。拟合误差有系统性的一面: 在期限不短于 240 天的合约上, 模型相对市场的平均误差在左翼为 -0.27 至 -0.40、右翼为 +0.25 至 +0.51 个波动率

点，校准得到的偏斜在长期限上比市场平缓。在拟合偏弱的三个快照（2021-12-01、2020-04-01、2024-06-03）上，较大的偏差同样出现在最短与最长的期限：2021-12-01 的 16 天期限上，市场左翼的隐含波动率高于 ξ 触及上界的曲线；2020-04-01 的 16 天期限上，平值附近的隐含波动率在 60% 至 91% 之间分散；2024-06-03 的 592 天期限上，市场左翼高于曲线 2-3 个点、右翼低于曲线约 1 个点。

以 120–450 天合约的 RMSE 为准，触及搜索边界的 8 个快照的均值为 0.84（中位数 0.65），其余 76 个为 0.70（0.59），差别不大。该 RMSE 超过 1.5 的快照有 6 个，全部在 2020 年 4 月至 9 月，其中只有 2020-04-01 被标为低可信度。按这一 RMSE 折算，校准不确定性的中位数为 0.58 ($K = S_t$) 与 0.39 ($K = 150$)，分别是蒙特卡罗标准误的 8.5 倍与 5.5 倍，是基准最主要的不确定性来源。

2 BSM 相对基准的表现

```
In [8]: def metrics(g, col, bench=BENCH):
    e = g[col] - g[bench]
    pct = e / g[bench] * 100
    return pd.Series({"天数": len(g), "MAE": e.abs().mean(), "RMSE": np.sqrt((e
        "平均偏差": e.mean(), "偏差中位数": e.median(),
        "MAPE_pct": pct.abs().mean(), "平均相对偏差_pct": pct.mean
        "P95绝对误差": e.abs().quantile(0.95)}))

versions = [("bsm_hist", "σ = 历史波动率"), ("bsm_impl", "σ = Heston 隐含")]
rows = []
for contract, g in df.groupby("contract"):
    subsets = [("全样本", g), ("仅可靠日期", g[g["reliable"]]), ("再剔除相关期限:
    for subset, h in subsets:
        for col, label in versions:
            rows.append(metrics(h.dropna(subset=[col, BENCH]), col).rename((cont
baseline = pd.DataFrame(rows)
baseline.index.names = ["合约", "样本", "BSM 版本"]
display(baseline)
baseline.to_csv(out / "bsm_baseline_metrics.csv", encoding="utf-8-sig")
print("偏差 = BSM - Heston, 为正表示 BSM 高估; 已保存 bsm_baseline_metrics.csv")

# 基准换成半解析精确值, 结果应几乎不变
main = [np.sqrt(((g["bsm_hist"] - g[BENCH]) ** 2).mean()) for _, g in df.groupby
alt = [np.sqrt(((g["bsm_hist"] - g["heston_exact"]) ** 2).mean()) for _, g in df
print("RMSE (σ=历史波动率): 以 MC 为基准", np.round(main, 4), "; 以精确值为基准"
```

			天数	MAE	RMSE	平均偏差	偏差中位数	MAPE_pct	平均相对偏差_pct
合约	样本	BSM 版本							
atm	全样本	$\sigma =$ 历史波动率	1761.0000	6.4372	9.4384	-1.2130	-3.2835	27.2002	-5.7317
		$\sigma =$ Heston 隐含	1761.0000	0.4112	0.5034	0.2974	0.3109	1.8460	1.3876
	仅可靠日期	$\sigma =$ 历史波动率	1573.0000	5.7423	7.4748	-2.0731	-3.3457	24.4271	-9.7885
		$\sigma =$ Heston 隐含	1573.0000	0.3846	0.4714	0.2756	0.2874	1.7870	1.3565
	再剔除相关期限拟合偏弱的快照	$\sigma =$ 历史波动率	1467.0000	5.6670	7.4450	-2.4072	-3.4121	24.1405	-11.2262
		$\sigma =$ Heston 隐含	1467.0000	0.3834	0.4721	0.2691	0.2731	1.7915	1.3396
k150	全样本	$\sigma =$ 历史波动率	1761.0000	4.4029	7.3252	-0.5454	-0.9384	13.0666	-3.9405
		$\sigma =$ Heston 隐含	1761.0000	1.1160	1.3446	0.4070	0.5303	3.2370	1.6855
	仅可靠日期	$\sigma =$ 历史波动率	1573.0000	3.6557	5.0863	-1.2365	-0.8652	11.8656	-5.2311
		$\sigma =$ Heston 隐含	1573.0000	1.1084	1.3371	0.4018	0.5323	3.2596	1.7364
	再剔除相关期限拟合偏弱的	$\sigma =$ 历史波动率	1467.0000	3.6413	5.0822	-1.4904	-0.9310	12.2318	-5.8990
		$\sigma =$ Heston 隐含	1467.0000	1.1563	1.3768	0.4114	0.5579	3.4383	1.8271

			天数	MAE	RMSE	平均偏差	偏差中位数	MAPE_pct	平均相对偏差_pct
合约	样本	BSM 版本							
	快照								

偏差 = BSM - Heston，为正表示 BSM 高估；已保存 bsm_baseline_metrics.csv
RMSE (σ =历史波动率)：以 MC 为基准 [9.4384 7.3252]；以精确值为基准 [9.4359 7.3251]

```
In [9]: # BSM 价格是确定的，它与 MC 价格之差的标准误就是 MC 的标准误
rows = []
for contract, g in df.dropna(subset=["bsm_hist", "bsm_impl", BENCH]).groupby("contract"):
    for col, label in versions:
        t = (g[col] - g[BENCH]).abs() / g["heston_mc_se"]
        rows.append({"合约": contract, "BSM 版本": label, "差距/标准误_中位数": t.median(),
                     "差距超过2倍标准误的占比": (t > 2).mean(), "差距超过3倍标准误的占比": (t > 3).mean()})
display(pd.DataFrame(rows).set_index(["合约", "BSM 版本"]))
```

合约	BSM 版本	差距/标准误_中位数	差距超过2倍标准误的占比	差距超过3倍标准误的占比
atm	σ = 历史波动率	78.0598	0.9898	0.9864
	σ = Heston 隐含	5.6782	0.7978	0.7110
k150	σ = 历史波动率	31.9845	0.9659	0.9415
	σ = Heston 隐含	13.0952	0.9364	0.9125

2.1 误差分解

```
In [10]: # 总误差 = 波动率估计误差 + 曲面形状误差
# bsm_hist - heston = (bsm_hist - bsm_impl) + (bsm_impl - heston)
rows = []
for contract, g in df.dropna(subset=["bsm_hist", "bsm_impl", BENCH]).groupby("contract"):
    for subset, h in [(("全样本", g), ("仅可靠日期", g[g["reliable"]]))]:
        parts = {"总误差": h["bsm_hist"] - h[BENCH],
                  "波动率估计误差": h["bsm_hist"] - h["bsm_impl"],
                  "曲面形状误差": h["bsm_impl"] - h[BENCH]}
        for name, e in parts.items():
            rows.append({"合约": contract, "样本": subset, "分量": name,
                         "平均": e.mean(), "RMSE": np.sqrt((e ** 2).mean())})
display(pd.DataFrame(rows).set_index(["合约", "样本", "分量"]))
```

		平均 RMSE	
合约	样本	分量	
atm	全样本	总误差	-1.2130 9.4384
		波动率估计误差	-1.5104 9.5415
		曲面形状误差	0.2974 0.5034
	仅可靠日期	总误差	-2.0731 7.4748
		波动率估计误差	-2.3487 7.5915
		曲面形状误差	0.2756 0.4714
k150	全样本	总误差	-0.5454 7.3252
		波动率估计误差	-0.9524 7.1149
		曲面形状误差	0.4070 1.3446
	仅可靠日期	总误差	-1.2365 5.0863
		波动率估计误差	-1.6382 4.9759
		曲面形状误差	0.4018 1.3371

```
In [11]: def bsm_legs(g, sigma):
    """BSM 下 chooser 的两条腿: Call(K, T2) 与  $e^{-q \tau}$  Put(K', T1)"""
    S, K, T1, T2 = (g[c].to_numpy() for c in ("S", "K", "T1", "T2"))
    r, q = g["r"].to_numpy(), g["q"].to_numpy()
    tau = T2 - T1
    Kp = K * np.exp(-(r - q) * tau)
    call = bsm_call(BSMParams(S, K, r, T2, sigma, q))
    put = np.exp(-q * tau) * bsm_put(BSMParams(S, Kp, r, T1, sigma, q))
    return call, put

rows = []
for contract, g in df.dropna(subset=["sigma_hist", "sigma_impl"]).groupby("contract"):
    for subset, h in [ ("全样本", g), ("仅可靠日期", g[g["reliable"]]) ]:
        for col, label in [ ("sigma_hist", "σ = 历史波动率"), ("sigma_impl", "σ = ") ]:
            sigma = h[col].to_numpy()
            ok = np.isfinite(sigma) & (sigma > 0)
            hh = h[ok]
            call, put = bsm_legs(hh, sigma[ok])
            for leg, bsm_v, hes_v in [ ("Call 腿", call, hh["heston_call_leg"].to_numpy()),
                                      ("Put 腿", put, hh["heston_put_leg"].to_numpy()) ]:
                e = bsm_v - hes_v
                rows.append({"合约": contract, "样本": subset, "BSM 版本": label, "腿": leg,
                            "平均偏差": e.mean(), "RMSE": np.sqrt((e ** 2).mean())})
display(pd.DataFrame(rows).set_index(["合约", "样本", "BSM 版本", "腿"])) #
```

合约	样本	BSM 版本	平均偏差		RMSE
			腿		
atm	全样本	$\sigma =$ 历史波动率	Call 腿	-0.9041	5.5969
			Put 腿	-0.3141	3.8530
		$\sigma =$ Heston 隐含	Call 腿	-0.0069	0.1642
			Put 腿	0.2992	0.4461
	仅可靠日期	$\sigma =$ 历史波动率	Call 腿	-1.4081	4.4749
			Put 腿	-0.6694	3.0138
		$\sigma =$ Heston 隐含	Call 腿	-0.0269	0.1560
			Put 腿	0.2982	0.4252
k150	全样本	$\sigma =$ 历史波动率	Call 腿	-0.3771	4.6789
			Put 腿	-0.1713	2.6726
		$\sigma =$ Heston 隐含	Call 腿	0.1823	0.7830
			Put 腿	0.2217	0.6243
	仅可靠日期	$\sigma =$ 历史波动率	Call 腿	-0.8217	3.2443
			Put 腿	-0.4171	1.8738
		$\sigma =$ Heston 隐含	Call 腿	0.1763	0.7750
			Put 腿	0.2230	0.6215

BSM 的表现. 以 20 日历史波动率为输入，BSM 相对基准的 RMSE 为 9.44 ($K = S_t$) 与 7.33 ($K = 150$)，MAPE 为 27.2% 与 13.1%，平均偏差为 -1.21 与 -0.55 ，中位数偏差为 -3.28 与 -0.94 ，均值高于中位数是因为高波动时期的高估拉高了均值；仅用可靠日期时 RMSE 为 7.47 与 5.09。以 Heston 在 T_2 期限、远期平值处的隐含波动率为输入，RMSE 降至 0.50 与 1.34，MAPE 为 1.85% 与 3.24%，平均偏差为 $+0.30$ 与 $+0.41$ ，仅用可靠日期时 RMSE 为 0.47 与 1.34。总误差以波动率估计误差为主（RMSE 9.54 与 7.11），曲面形状误差的 RMSE 为 0.50 与 1.34。 $K = S_t$ 合约的形状误差集中在 Put 腿（平均 $+0.30$ ），Call 腿接近 0，这是构造所致，因为隐含波动率取自 Call 腿所在位置； $K = 150$ 合约两条腿的偏差均为正（ $+0.18$ 与 $+0.22$ ）。基准换成半解析精确值时，RMSE 的变化在小数点后第四位。以蒙特卡罗标准误计，两个版本与基准的差距中位数分别为 78 倍与 32 倍（历史波动率）、5.7 倍与 13 倍（Heston 隐含），都远超抽样噪声。

2.2 误差来源

```
In [12]: sets = {"各参数的中位数": hp[PN].median().to_dict()}
for dt, label in [("2021-12-01", "ξ 撞界"), ("2024-06-03", "κ 撞界"), ("2020-04-
    sets[f"{dt} ({label}) "] = hp.loc[pd.Timestamp(dt), PN].astype(float).to_dic

pd.DataFrame(sets).T.round(3)
```

Out[12]:

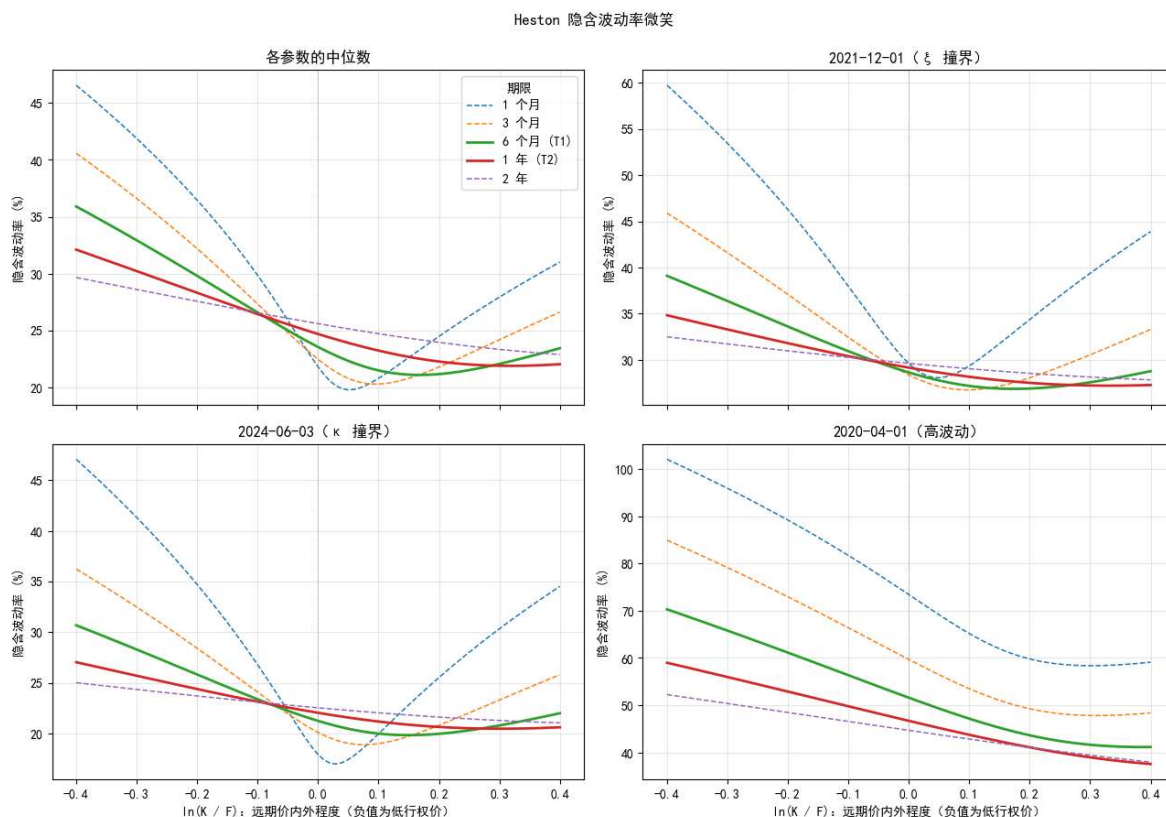
	v0	kappa	theta	xi	rho
各参数的中位数	0.0540	7.5140	0.0770	1.8550	-0.5030
2021-12-01 (ξ 撞界)	0.1410	17.5180	0.0960	4.0000	-0.3840
2024-06-03 (κ 撞界)	0.0320	25.0000	0.0550	3.4780	-0.3890
2020-04-01 (高波动)	0.7210	4.0340	0.2800	4.0000	-0.6820

```

In [13]: ks = np.linspace(-0.4, 0.4, 81)
maturities = [(1 / 12, "1 个月"), (0.25, "3 个月"), (0.5, "6 个月 (T1)"), (1.0,

fig, axes = plt.subplots(2, 2, figsize=(13, 9), sharex=True)
for ax, (name, p) in zip(axes.flat, sets.items()):
    surf = iv_surface(p, ks, np.array([t for t, _ in maturities]))
    for row, (t, label) in zip(surf, maturities):
        main_leg = t in (0.5, 1.0)
        ax.plot(ks, row * 100, label=label, lw=2 if main_leg else 1.1, ls="-" if
ax.axvline(0, color="gray", lw=0.6, ls=":")
ax.set(title=name, ylabel="隐含波动率 (%)")
ax.grid(alpha=0.3)
axes[0, 0].legend(title="期限")
for ax in axes[1]:
    ax.set_xlabel("ln(K / F): 远期价内外程度 (负值为低行权价)")
fig.suptitle("Heston 隐含波动率微笑", y=0.995)
plt.tight_layout()
plt.show()

```



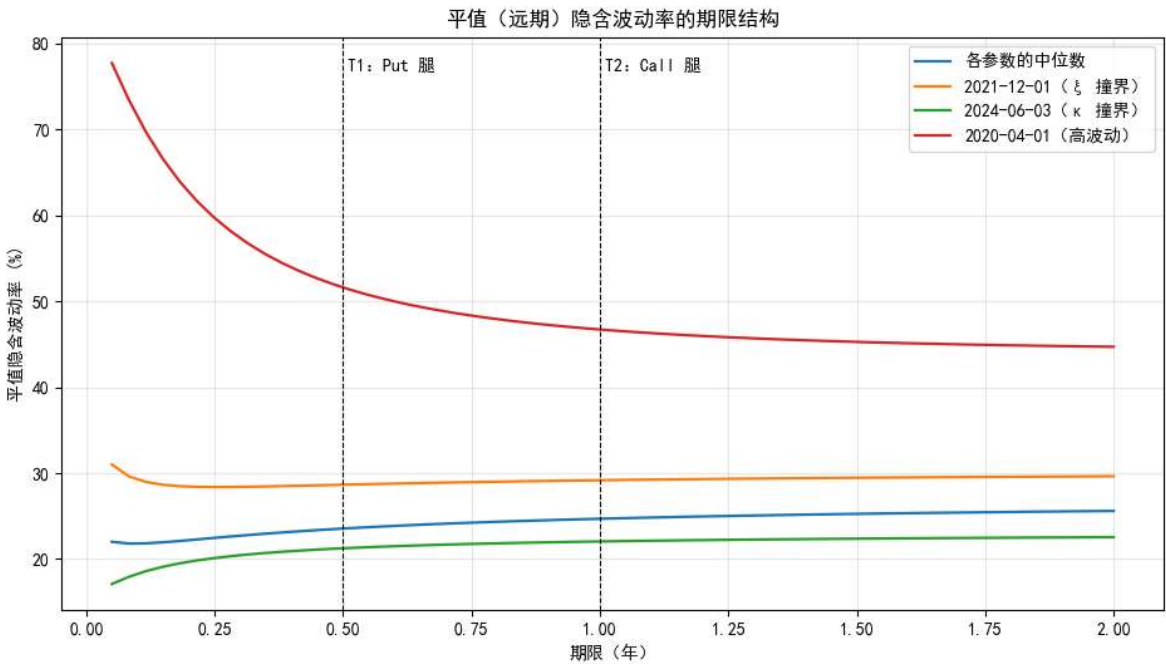
```

In [14]: Ts = np.linspace(0.05, 2.0, 60)
fig, ax = plt.subplots(figsize=(9.5, 5.5))
for name, p in sets.items():
    ax.plot(Ts, iv_surface(p, np.array([0.0]), Ts[:, 0]) * 100, label=name, lw=1)
top = ax.get_ylim()[1]
for T, label in [(0.5, "T1: Put 腿"), (1.0, "T2: Call 腿")]:

```

```
ax.axvline(T, color="k", ls="--", lw=0.8)
ax.text(T + 0.01, top * 0.97, label, va="top")
ax.set(xlabel="期限 (年)", ylabel="平值隐含波动率 (%)", title="平值 (远期) 隐含波动率的期限结构")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# 两条腿的远期价内外程度相同, 只有期限不同, 所以  $\sigma_P - \sigma_C$  就是这条期限结构在 T1
tbl = pd.DataFrame({
    name: {" $\sigma(T1=0.5)$ ": iv_surface(p, np.array([0.0]), np.array([0.5]))[0, 0] *
        " $\sigma(T2=1)$ ": iv_surface(p, np.array([0.0]), np.array([1.0]))[0, 0] * 10
    for name, p in sets.items()}).T
tbl["P-C (波动率点)"] = tbl[" $\sigma(T1=0.5)$ "] - tbl[" $\sigma(T2=1)$ "]
tbl.round(2)
```



Out[14]:

	$\sigma(T1=0.5)$	$\sigma(T2=1)$	P-C (波动率点)
各参数的中位数	23.5300	24.6800	-1.1400
2021-12-01 (ξ 撞界)	28.6400	29.1600	-0.5200
2024-06-03 (κ 撞界)	21.2400	22.0400	-0.8000
2020-04-01 (高波动)	51.6100	46.7100	4.8900

```
In [15]: # 逐日的两条腿隐含波动率, 单位为波动率点
dd = df.dropna(subset=["sigma_impl", "sigma_hist"]).copy()
r, q = dd["r"].to_numpy(), dd["q"].to_numpy()
tau = (dd["T2"] - dd["T1"]).to_numpy()
Kp = dd["K"].to_numpy() * np.exp(-(r - q) * tau)
put_price = dd["heston_put_leg"].to_numpy() / np.exp(-q * tau) # 还原为 Put

dd["sigma_C"] = implied_vol(dd["heston_call_leg"].to_numpy(), dd["S"].to_numpy(),
                             r, dd["T2"].to_numpy(), q, True)
dd["sigma_P"] = implied_vol(put_price, dd["S"].to_numpy(), Kp, r, dd["T1"].to_numpy())
dd["C-impl"] = (dd["sigma_C"] - dd["sigma_impl"]) * 100
dd["P-C"] = (dd["sigma_P"] - dd["sigma_C"]) * 100
dd["hist-impl"] = (dd["sigma_hist"] - dd["sigma_impl"]) * 100
```

```
display(dd.groupby("contract")[["C-impl", "P-C", "hist-impl"]].agg(["mean", "median", "min", "max", "std"]))
print("C-impl: Call 腿位置的隐含波动率减  $\sigma_{impl}$ ; P-C: Put 腿减 Call 腿; hist-impl: 历史波动率减  $\sigma_{impl}$ ")

atm = dd[dd["contract"] == "atm"].assign(year=lambda x: x["date"].dt.year)
by_year = atm.groupby("year")["P-C"].agg(["mean", "median", "min", "max"])
by_year["P-C>0 的天数占比"] = atm.groupby("year")["P-C"].agg(lambda s: (s > 0).sum())
display(by_year.round(2))
```

contract	C-impl			P-C			hist-impl		
	mean	median	std	mean	median	std	mean	median	std
atm	-0.0238	-0.0748	0.3174	-0.7738	-0.9465	0.9734	-1.0225	-4.2124	13.079
k150	-0.8754	-1.4015	2.8871	0.1108	-0.4631	1.8862	-1.0225	-4.2124	13.079

C-impl: Call 腿位置的隐含波动率减 σ_{impl} ; P-C: Put 腿减 Call 腿; hist-impl: 历史波动率减 σ_{impl}

year	mean	median	min	max	P-C>0 的天数占比
2018	-0.5500	-0.7500	-2.1100	5.3500	0.1800
2019	-1.2800	-1.2700	-2.2700	0.3300	0.0100
2020	-0.0500	-0.6400	-2.1600	5.4200	0.2800
2021	-1.2600	-1.3400	-1.8900	0.3200	0.0000
2022	-0.6700	-0.8300	-1.8100	2.2900	0.1700
2023	-1.0600	-1.1700	-2.0600	0.7200	0.0700
2024	-0.5400	-0.6300	-1.3100	1.1800	0.1600

曲面与两条腿. 校准得到的 Heston 隐含波动率曲面具有负偏斜 (ρ 在 -0.69 至 -0.19)，微笑的曲率随期限衰减。chooser 的两条腿位于曲面上同一远期价内外程度， $\ln(K/F(T_2)) = \ln(K'/F(T_1)) = \ln(K/S) - (r - q)T_2$ ，只有期限不同，所以两条腿隐含波动率之差只反映该位置的期限结构。在 2021-12-01、2024-06-03 与各参数中位数的快照上， T_1 与 T_2 的平值隐含波动率之差 (Put 腿减 Call 腿) 为 -0.52 、 -0.80 与 -1.14 个波动率点，期限结构向上；在期限结构倒挂的 2020-04-01 为 $+4.89$ 。BSM 以单一波动率为两条腿定价，期限结构向上时 Put 腿偏高，倒挂时偏低，这与 $K = S_t$ 合约 Put 腿的平均偏差为正 ($+0.30$) 的符号相符。

3 失效模式

```
In [16]: g = df[df["contract"] == "atm"].sort_values("date")
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True, gridspec_kw={"height_ratios": (1, 1)})

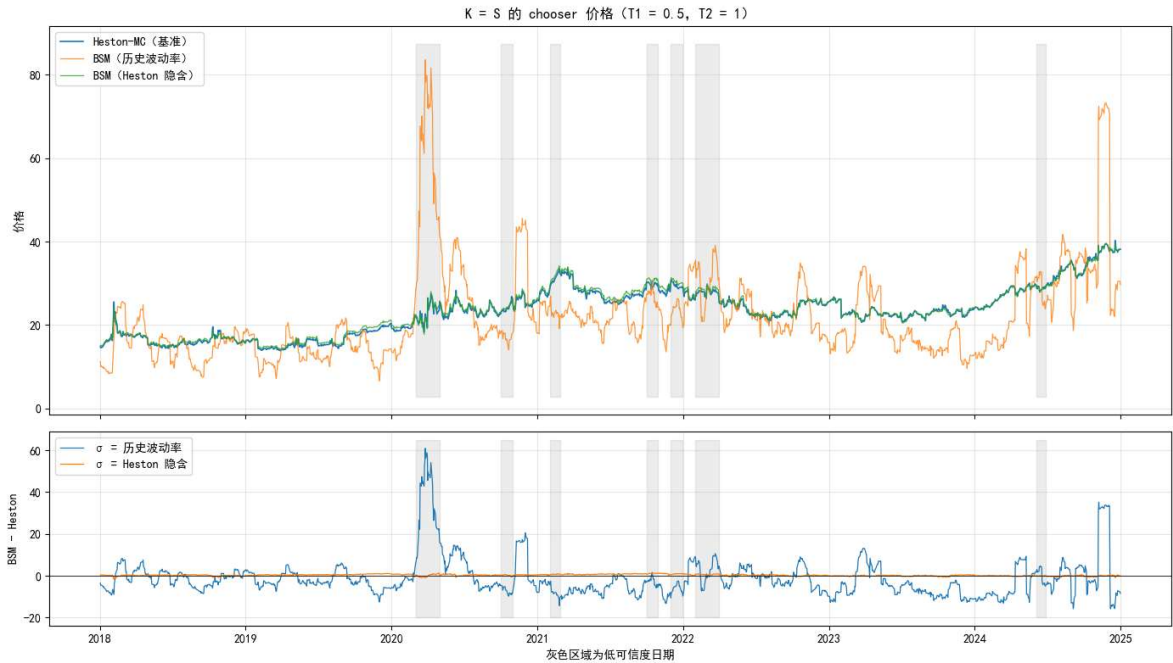
axes[0].plot(g["date"], g[BENCH], label="Heston-MC (基准)", lw=1.2)
axes[0].plot(g["date"], g["bsm_hist"], label="BSM (历史波动率)", lw=0.9, alpha=0.5)
axes[0].plot(g["date"], g["bsm_impl"], label="BSM (Heston 隐含)", lw=0.9, alpha=0.5)
axes[0].set(title="K = S 的 chooser 价格 (T1 = 0.5, T2 = 1)", ylabel="价格")
axes[0].legend()
```

```

axes[1].plot(g["date"], g["bsm_hist"] - g[BENCH], label="σ = 历史波动率", lw=0.9)
axes[1].plot(g["date"], g["bsm_impl"] - g[BENCH], label="σ = Heston 隐含", lw=0.9)
axes[1].axhline(0, color="k", lw=0.6)
axes[1].set_ylabel("BSM - Heston")
axes[1].legend()

for ax in axes:
    ax.fill_between(g["date"], *ax.get_ylim(), where=~g["reliable"].to_numpy(),
                   alpha=0.3)
axes[1].set_xlabel("灰色区域为低可信度日期")
plt.tight_layout()
plt.show()

```



```
In [17]: df["vix_bin"] = pd.cut(df["vix"], bins=[0, 15, 20, 30, 1000], labels=["<15", "15
```

```

def by_group(keys):
    rows = []
    for k, g in df.dropna(subset=["bsm_hist", "bsm_impl", BENCH]).groupby(keys,
                                   k = k if isinstance(k, tuple) else (k,))
        a, b = metrics(g, "bsm_hist"), metrics(g, "bsm_impl")
        rows.append(dict(zip(["合约"] + (["分组"] if len(k) == 2 else []), k)) |
                      {"天数": a["天数"], "RMSE(hist)": a["RMSE"], "偏差(hist)": a
                       "RMSE(impl)": b["RMSE"], "偏差(impl)": b["平均偏差"], "MAPE
    return pd.DataFrame(rows)

display(by_group(["contract", "vix_bin"]).set_index(["合约", "分组"]))

```

		天数	RMSE(hist)	偏差 (hist)	MAPE% (hist)	RMSE(impl)	偏差 (impl)	MAPE% (impl)
合约	分组							
atm	<15	510.0000	8.2744	-3.9068	28.2239	0.4484	0.3080	1.9972
	15-20	572.0000	7.0844	-2.1547	22.5356	0.5641	0.3565	1.9662
	20-30	539.0000	6.8133	-1.1237	21.9563	0.4808	0.2923	1.6044
	≥30	140.0000	22.0761	12.1041	62.7176	0.5147	0.0372	1.7344
k150	<15	510.0000	4.0235	-2.3450	9.7650	1.4494	-0.0258	3.6707
	15-20	572.0000	5.3122	-2.1783	13.8585	1.2059	0.1947	2.9014
	20-30	539.0000	5.8727	-0.1076	13.1159	1.3564	0.8425	3.2710
	≥30	140.0000	19.1806	10.9960	21.6683	1.4371	1.1741	2.8981

```
In [18]: events = {"2020 年 3-4 月（疫情）": ("2020-02-20", "2020-04-30"),
                  "2022 年（加息周期）": ("2022-01-01", "2022-12-31"),
                  "2023 年 3-4 月（银行业危机）": ("2023-03-01", "2023-04-15")}
df["event"] = "其他"
for name, (start, end) in events.items():
    df.loc[df["date"].between(start, end), "event"] = name

display(by_group(["contract", "event"]).set_index(["合约", "分组"]))
```

		天数	RMSE(hist)	偏差 (hist)	MAPE% (hist)	RMSE(impl)	偏差 (impl)	MAPE% (impl)
合约	分组							
atm	2020年3-4月(疫情)	50.0000	35.6212	30.1042	134.0478	0.6726	0.0778	2.6560
	2022年(加息周期)	251.0000	4.8763	-0.1992	15.8742	0.4364	0.3038	1.4364
	2023年3-4月(银行业危机)	32.0000	8.8153	5.2773	36.4404	0.1585	0.0539	0.6379
	其他	1428.0000	7.7141	-2.6331	25.2427	0.5125	0.3094	1.9167
k150	2020年3-4月(疫情)	50.0000	31.2647	25.9403	43.0366	1.3960	1.2893	2.7009
	2022年(加息周期)	251.0000	4.7472	1.4491	12.3577	1.8525	1.6782	5.3799
	2023年3-4月(银行业危机)	32.0000	9.7731	6.7697	35.1244	1.9786	1.9502	7.6580
	其他	1428.0000	5.0837	-1.9873	11.6475	1.2126	0.1181	2.7801



```
In [19]: k150 = df[df["contract"] == "k150"].copy()
k150["S/K"] = k150["S"] / k150["K"]
k150["moneyness"] = pd.cut(k150["S/K"], [0, 0.7, 0.9, 1.1, 1.4, 10], right=False,
                             labels=["<0.7", "0.7-0.9", "0.9-1.1", "1.1-1.4", ">1.4"])
rows = []
for m, g in k150.dropna(subset=["bsm_hist", "bsm_impl", "BENCH"]).groupby("moneyness"):
    a, b = metrics(g, "bsm_hist"), metrics(g, "bsm_impl")
    rows.append({"S/K": m, "天数": a["天数"], "RMSE(hist)": a["RMSE"], "偏差(hist)": a["MAPE_pct"], "RMSE(impl)": b["RMSE"], "偏差(impl)": b["MAPE_pct"]})
display(pd.DataFrame(rows).set_index("S/K"))
```

	天数	RMSE(hist)	偏差 (hist)	MAPE% (hist)	RMSE(impl)	偏差 (impl)	MAPE% (impl)
S/K							
<0.7	250.0000	14.2662	5.6785	11.1667	0.6887	0.4665	0.9767
0.7-0.9	690.0000	4.3378	1.2531	7.7135	1.3732	1.1688	3.5463
0.9-1.1	510.0000	6.8823	-4.5661	23.6186	1.1201	0.7058	3.6168
1.1-1.4	216.0000	5.1286	-4.0551	11.8725	1.9770	-1.8996	4.4524
≥1.4	95.0000	2.8827	-0.4226	3.0138	1.7180	-1.6429	2.1371

失效模式. 历史波动率版本的偏差随市场状态改变符号： $K = S_t$ 合约在 VIX 低于 15 时平均低估 3.91 (MAPE 28.2%)，在 VIX 不低于 30 时平均高估 12.10 (MAPE 62.7%)；2020 年 3-4 月 RMSE 为 35.6、MAPE 为 134%，2023 年 3-4 月 RMSE 为 8.8。历史波动率的价格在 2020 年 3 月 (约 83) 与 2024 年 11-12 月 (约 73) 出现尖峰。隐含波动率版本的 RMSE 在各 VIX 分档中保持在 0.45-0.56 ($K = S_t$) 与 1.21-1.45 ($K = 150$)； $K = 150$ 合约在 2022 年与 2023 年 3-4 月升至 1.85 与 1.98，高于其余时期的 1.21。 $K = 150$ 合约的形状误差随 S/K 变号： $S/K < 1.1$ 时为 +0.47 至 +1.17， $S/K \geq 1.1$ 时为 -1.90 与 -1.64。

4 情绪缺口

```
In [20]: # 误差与情绪的相关性，只是相关，两个模型都不含情绪
s = df.dropna(subset=["sentiment_score", "vix", "bsm_hist", BENCH]).copy()
s["rel_err"] = (s["bsm_hist"] - s[BENCH]) / s[BENCH] * 100
s["abs_rel_err"] = s["rel_err"].abs()
print(f"有情绪分的交易日占比: {df['sentiment_score'].notna().mean():.1%}")

def rank_corr(x, y):
    return np.corrcoef(rankdata(x), rankdata(y))[0, 1]

def partial_rank_corr(x, y, z):
    """秩的偏相关：把 x、y 的秩对 z 的秩回归，取残差再算相关"""
    rx, ry, rz = rankdata(x), rankdata(y), rankdata(z)
    resid = lambda a: a - np.polyval(np.polyfit(rz, a, 1), rz)
    return np.corrcoef(resid(rx), resid(ry))[0, 1]

rows = []
for contract, g in s.groupby("contract"):
    for target in ["rel_err", "abs_rel_err"]:
        for var in ["sentiment_score", "news_count"]:
            gg = g.dropna(subset=[var])
            rows.append({"合约": contract, "误差": target, "变量": var, "天数":
                "秩相关": rank_corr(gg[var], gg[target]),
                "控制 VIX 后的偏相关": partial_rank_corr(gg[var], gg[target], s[BENCH])})
display(pd.DataFrame(rows).set_index(["合约", "误差", "变量"]))
```

```
s["情绪分组"] = s.groupby("contract")["sentiment_score"].transform(lambda x: pd.
display(s.groupby(["contract", "情绪分组"], observed=True)
.agg(天数=("rel_err", "size"), 平均相对偏差_pct=("rel_err", "mean"), 平均
```

有情绪分的交易日占比：86.9%

		天数	秩相关	控制 VIX 后的偏相关
合约	误差	变量		
atm	rel_err	sentiment_score	1531	-0.0263
		news_count	1531	-0.0778
	abs_rel_err	sentiment_score	1531	-0.0037
		news_count	1531	-0.0237
k150	rel_err	sentiment_score	1531	-0.0545
		news_count	1531	-0.1872
	abs_rel_err	sentiment_score	1531	0.0229
		news_count	1531	0.1533

		天数	平均相对偏差_pct	平均绝对相对偏差_pct
contract	情绪分组			
atm	低	511	-3.9551	27.6344
	中	510	-6.7005	26.9727
	高	510	-8.5864	25.4181
k150	低	511	-2.7548	13.0831
	中	510	-5.1991	14.1133
	高	510	-5.4768	13.4645

情绪缺口. 在 1531 个有情绪分的交易日上，误差与情绪分的秩相关在 -0.05 到 $+0.02$ 之间；控制 VIX 后，情绪分与有符号误差的偏相关为 -0.01 ($K = S_t$) 与 -0.04 ($K = 150$)，情绪分在 VIX 之外对误差没有解释力。与新闻篇数的偏相关分别为 -0.13 与 -0.24 (有符号误差)、 -0.01 与 $+0.14$ (绝对误差)，幅度较小。按情绪分三等分， $K = S_t$ 的平均相对偏差由 -4.0% 经 -6.7% 到 -8.6% ，平均绝对相对偏差由 27.6% 降至 25.4% ，几乎不随情绪变化。以上均为相关性，两个模型都不含情绪；日度序列自相关强，有效样本远少于 1531 天，未给出显著性检验。

5 总结

本节以与市场期权价格一致的 Heston 模型为基准，检验常数波动率的 BSM 对 chooser 期权的定价。基准的蒙特卡罗误差（标准误差约 0.07，离散偏差约 0.005）远小于其校准误差（折算为价格约 0.4–0.6），后者决定了差异能否被解读为模型差异。BSM 的表现取决于波动率输入：以历史波动率为输入时 RMSE 为 9.44 与 7.33，偏差随 VIX 变号，2020 年 3–4 月达到 35.6；以 Heston 隐含波动率为输入时降至 0.50 与 1.34。总误差的主要部分来自波动率估计，常数波动率假设带来的曲面形状误差较小： $K = S_t$ 合约的绝对差中位数 (0.36) 小于折算的校准不确定性 (0.58)，仅 30% 的交易日超过它，无法与之

区分； $K = 150$ 合约的 (0.95) 大于校准不确定性 (0.39)，81% 的交易日超过它，可以辨别。在控制 VIX 之后，没有发现误差与情绪分的相关。上述比较的是 BSM 与 Heston 的模型差异，不是对市场价格的误差，因为 chooser 没有市场成交价。

附录 Heston 隐含波动率曲面

```
In [21]: name, p = list(sets.items())[0]
kk = np.linspace(-0.4, 0.4, 41)
TT = np.linspace(0.08, 2.0, 40)
Z = iv_surface(p, kk, TT) * 100
KK, T_ = np.meshgrid(kk, TT)

fig = plt.figure(figsize=(10.5, 7.2))
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(KK, T_, Z, cmap=cm.viridis, alpha=0.8, linewidth=0, antialiased=
for T, label in [(0.5, "Put 腿 (T1 = 0.5)"), (1.0, "Call 腿 (T2 = 1)"]):
    z = iv_surface(p, np.array([0.0]), np.array([T]))[0, 0] * 100
    ax.plot([0, 0], [T, T], [z, z + 9], color="red", lw=2, zorder=10)
    ax.scatter([0], [T], [z + 9], color="red", s=50, depthshade=False, zorder=11)
    ax.text(0.03, T, z + 10.5, label, color="red", zorder=12)
ax.set(xlabel="ln(K/F)", ylabel="期限 (年)", zlabel="隐含波动率 (%)", title=f"H
ax.view_init(elev=24, azimuth=-62)
plt.tight_layout()
plt.show()
```

Heston 隐含波动率曲面（各参数的中位数）

